

Report: Python to Rust Code Translation

1. Introduction

In this task, I used the HKGAI-V1 model to translate five Python programs into Rust. The purpose is to understand how AI can help in code translation and how different language structures cause difficulties.

I used OpenWebUI with temperature 0.0 and did every translation in a separate chat session.

2. Process Overview

For each program, I copied the correct Python source code from the given folder and pasted it into HKGAI-V1 with a short prompt:

“You are a code translator. Please translate this Python code into Rust, keeping the same logic and correctness.”

Or a longer one:

“You are a code translator. Please translate the following correct Python code into Rust, ensuring the same functionality and correctness. Keep the structure idiomatic for Rust (use Vec, Result, Option, etc. appropriately). Do not simplify or alter logic. Here is the Python code:...”

Then I compiled the generated Rust code on LeetCode using the Rust option. Usually the first version failed because of syntax or compile problems. I copied the compiler or runtime error back to the same chat and asked the model to fix it. After several rounds each problem finally passed all test cases.

3. Problems and Fixing

The most common issue was type mismatch or borrow checker errors.

For example, in *basic-calculator*, Rust complained about duplicate definition of Solution and wrong method call using &Solution. I learned to remove the extra argument and make struct used manually.

In *count-the-number-of-special-characters-ii*, the model first wrote a very slow version. After I told it about time complexity, it rewrote using index arrays for letters, which is more efficient.

In *minimum-cost-good-caption*, the memoization function caused “cannot move out of borrowed reference” error. I fixed it by returning `res.clone()` instead of moving the value.

In *robot-collisions*, many compile errors happened because of mutable variables. I

reminded the model to add `mut` in for `mut robot` in `vals`. Then I asked for further optimization to reduce sorting and data copying. Finally the model produced a better $O(n)$ version using indexes.

For *sum-of-largest-prime-substrings*, the first version overflowed on `i32`. I asked the model to use `i64`, which solved the runtime limit error.

4. LLM Performance and Reflection

The model translated structure and logic quite well but often failed small Rust details like borrowing and ownership. It does not fully understand the lifetime or move rules. Also, it sometimes simplified or changed logic, which I needed to check carefully. However, its ability to fix its own mistakes after I provided the compiler error was impressive.

For optimization, it required human guidance: only after I mentioned “too slow” or “ $O(n \log n)$ ” it tried to improve complexity. So human reasoning is still important.

5. Conclusion

This task showed that LLM can be a strong helper for code translation but not a perfect one. It can quickly give working drafts and fix compiler feedback, yet it still needs human checking for correctness and efficiency. Through the process I learned more about Rust syntax, ownership, and how to guide AI with clear instructions. The combination of human review and AI assistance is the best strategy for multi-language programming tasks.